

A Service Infrastructure for Industrie 4.0 Testbeds based on Asset Administration Shells

Andreas Luxenburger, Daniel Porta, Sönke Knoch, Jonas Mohr, and Tim Schwartz

Cognitive Assistants

German Research Center for Artificial Intelligence (DFKI)

Saarbrücken, Germany

{firstname.lastname}@dfki.de

Abstract—Industrial testbeds are spawning all across Europe, focusing on diverse topics. In our Human-Robot-Collaboration lab, we are researching how the lives of people in production can be made simpler through the use of collaborative robots, but still safe against the backdrop of the necessary changeability. Standardized digital twins based on Asset Administration Shells embedded in a service-oriented architecture are an ideal underlying information technology for this endeavor. In this paper, we describe such an infrastructure and outline its use based on two different use cases. We are convinced that this is not only relevant for testbed operators who want to achieve technical and semantic interoperability of their facilities, but can also serve as a blueprint for the implementation of Industrie 4.0 in small and medium-sized enterprises.

Index Terms—Asset Administration Shell, Digital Twins, Industrie 4.0, Service Infrastructure

I. INTRODUCTION

The digitalization of the industrial sector and the applications based on it within the context of Industrie 4.0 (I4.0) require a standardized information technological basis that makes it possible to describe data and information in a uniform manner and make it available across companies. The Asset Administration Shell (AAS) [1] is an important key technology here, enabling asset-specific data to be captured across the entire lifecycle and across all facets of the Reference Architecture Model I4.0 (RAMI4.0) in different submodels and made available for applications on a file or service basis by a standardized API [2]. In this way, a common understanding of facts can be expressed and exchanged when collaborating with partners, e.g. along a value chain.

Rapid digitalization and appropriation of I4.0 concepts beyond the early adopters, i.e., especially by small and medium-sized enterprises (SMEs), is necessary in order to be able to meet future market requirements, e.g. with regard to changeable and shared production or resilience. In our opinion, industrial testbeds, which have been set up successfully in large numbers in recent years with different focal points, play a special role in this transfer, as they often serve as regional contact points for SMEs.

The German-Czech Innovation Laboratory for Human-Robot Collaboration in Industrie 4.0 (Deutsch-Tschechisches Innovationslabor für Mensch-Roboter Kollaboration in Industrie 4.0), or short the MRK 4.0 lab, was established in 2016. It is operated by our research department and is part of

the Power4Production Center for Innovative Production Technologies (<https://power4production.de/en/home-english/>). Additional application domains apart from Industrie 4.0 include Retail and Medical Institutions. The lab operates more than 20 collaborative robotic systems (cobots) of different types like e.g. single- and dual-arm robots for pick&place tasks, mobile systems such as autonomous mobile robots (AMR) and automated guided vehicles (AGV) for transportation or drones for inspection. Among the manufacturers of the cobots are well-known representatives from the industry like Universal Robots (UR), KUKA, Franka Emika and Mobile Industrial Robots. In order to establish a bi-directional communication between humans and their robotic colleagues, we use multi-modal multisensor-systems including RGB-D cameras, motion capturing systems, high-precision laser scanners, laser projectors and various Extended Reality (XR) devices including Augmented Reality (AR) and Virtual Reality (VR) glasses covering use-cases of the Industrial Metaverse, in particular remote operations such as remote teach-in and remote human-human collaboration with robotic embodiment.

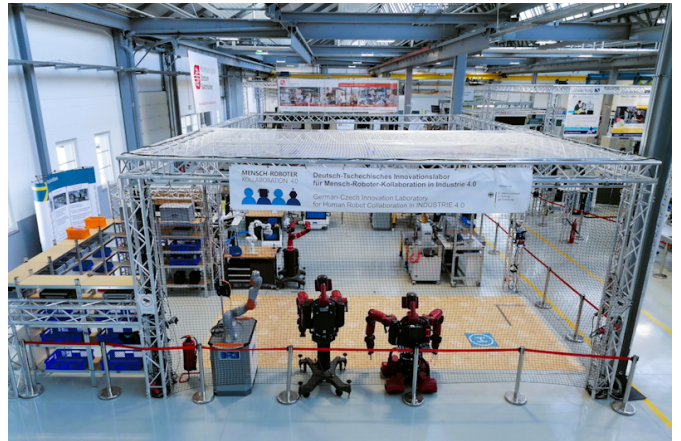


Fig. 1. DFKI's German-Czech Innovation Laboratory for Human-Robot Collaboration in Industrie 4.0

In this respect, several demonstration systems have been developed in the context of research and industry projects together with domain experts from the industry covering AR-based assistance systems for the orchestration of hybrid teams in aircraft assembly [3], cognitive social cobots with intent and

activity recognition of human workers in order to assist them in unergonomic, repetitive tasks [4], as well as applications and approaches for remote applications [5], [6].

Since the beginning of the BaSys project series (<https://www.basys40.de/>) in 2016, we have also been using the lab as an real-world experimentation facility for the implementation of an AAS-based service infrastructure and its application to build further industrial demonstration systems.

When interacting with representatives of SMEs throughout our transfer activities they often like and show high interest in our demonstrations. At the same time, they express their desire for standardization as well as for easy and low-cost solutions to achieve the necessary I4.0 maturity level [7] beforehand.

When it comes to collaboration on a European and international scale, e.g. through research projects like RICAIP (<https://ricaip.eu>), where different testbeds are to be connected on a technical level, an AAS-based infrastructure (together with other technology like the Eclipse Data Space Connector or Gaia-X which is however out-of-scope for this paper) provides a huge opportunity. On the one hand, reaching common goals can be accomplished more efficiently when operating on a common infrastructure. On the other hand, deploying AAS-based infrastructures on partner testbeds and demonstrating them the advantages will serve as a multiplier for the adoption of the AAS in general.

In this paper, with focus on the use cases of adaptive online planning of production processes and functional safety, we describe in detail the architecture and use of our AAS-based infrastructure in terms of relevant submodels and service components in order to implement these use cases. This way, we demonstrate how they work and give practical examples for interested partners in research and industry. The infrastructure is based on prior conceptual work on architecture blueprints to enable scalable vertical integration of assets with digital twins [8].

The paper is outlined as follows. First, we describe our use cases in Section II for the implementation of which we used our core AAS-based service infrastructure as described in Section III. Finally, we conclude in Section IV.

II. USE CASES

In our BaSys satellite projects BaSynaos and BaSySafe we focussed on (1) online-planning for intralogistics and manufacturing and (2) digital twins for functional safety. In this section, we will introduce these use cases and derive the required AAS submodels and service components for their realization.

A. Online-Planning for Intralogistics and Manufacturing

A central aspect of I4.0 is a seamless fusion of intralogistics and manufacturing as essential components of the value chain. For higher-level orchestration systems such as route and process planners, a continuous exchange of information between involved assets on the logistics side and on the manufacturing side is indispensable in order to coordinate the respective own optimization processes and thus achieve a

higher value stream in the factory. In order to ensure a smooth running of production processes which have to be adapted to current, potentially high frequently changing conditions on the shopfloor (especially in highly dynamic HRC scenarios), online planning systems require on the one hand fast access to continuously available planning-relevant information both before and during production. Such continuous (near) real-time data streams that are produced by assets are described in a dedicated signal submodel. On the other hand, suitable operational interfaces are needed for the control of involved assets, which should ideally be manufacturer-independent in order to reduce integration and commissioning costs. For that, we employ control components¹ with its standardized submodels.

In the initialization phase of the online planning system, information about the overall structure of the production site is necessary to bootstrap an internal representation. In particular, topology and layout submodels were used to describe hierarchical relationships between different production areas, manufacturing and logistics units, and storage areas, as well as corresponding access points and route networks for different types of AGVs. The latter were provided in the form of navigation graphs incorporating the VDA5050 communication standard (<https://github.com/VDA5050/VDA5050>) to describe the communication between AGVs and a master control via the layout submodel of our lab. Furthermore, asset type descriptive submodels with references to corresponding type AAS were provided to the online planner, including submodels describing technical specifications, such as vehicle dimensions, critical charging state, or speed profiles. The uniform description of this information by standardized submodels of the AAS results in a significant reduction of effort when importing different but uniformly described vehicle types.

Products to be manufactured are also reflected in a product type AAS which, in addition to the structure of the product (BOM), describes the corresponding manufacturing process. Capabilities requested therein, such as joining or picking up and placing certain product parts, are matched by the planner before and during production with the capabilities of declared manufacturing units via their AAS. Fig. 2 shows an example of these submodels in front of the implemented project demonstrator.

The execution of requested capabilities of an asset is performed by issuing requests for corresponding skills of the asset's control component, which are declared in corresponding control component submodels and linked in the asset's capability submodel. An example is shown in Fig. 3, where a VDA5050-based transport task is mapped to a corresponding skill request ("Move to symbolic position") of a real-world AMR's control component.

To avoid spending a lot of time on developing real robotic skills, which was not the focus of the project, we use the simulation feature of the control component in combination

¹https://wiki.eclipse.org/BaSysx/_/Documentation/_/API/_/ControlComponent

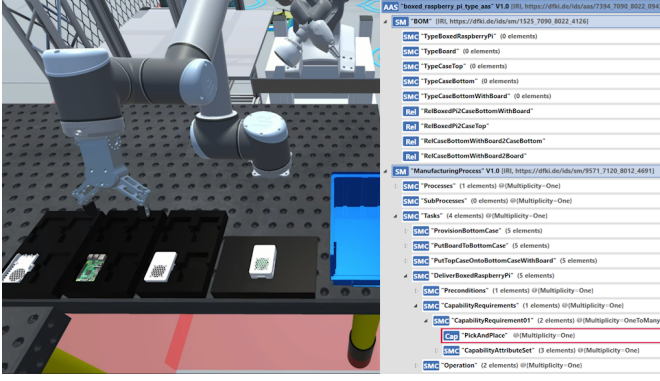


Fig. 2. Simulated product assembly: Online planners can select appropriate assets (e.g. UR10 robot) by accessing provided product, process and capability submodels and bring requested steps of the manufacturing process to execution.

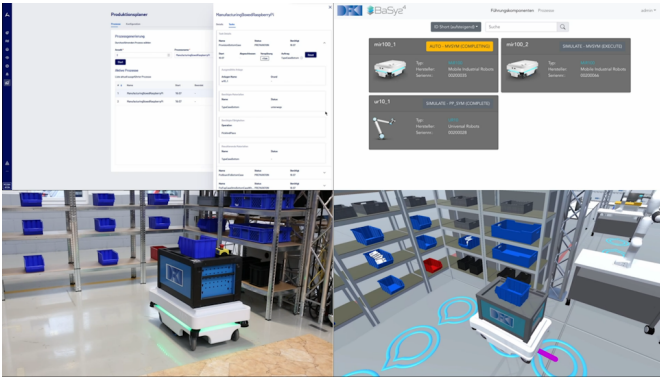


Fig. 3. Execution of a partially simulated production process using our proposed service infrastructure. **First row:** (a) Planner dashboard with two requested instances of a production process and related steps. (b) Control component states of involved assets. **Second row:** (c) Real-world AMR (MiR100) approaches target shelf for picking parts. (d) Corresponding state in digital twin with simulated product parts and pick&drop actions.

with our Unity3D-based visualization and simulation environment also shown in Fig. 3 and Fig. 2. Certain skills like picking raw materials from a target container in a shelf or dropping finalized products have not yet been implemented for the real asset but only exist in the virtual world. However, these simulated skills produce data just like the real-world skills would do realizing a virtual commissioning utilizing digital twins.

B. Digital Twins for Functional Safety

In the BaSys satellite BaSySafe a digital twin for functional safety was developed. Therefore, a safety submodel was created to support both the risk assessment during the engineering phase and the monitoring of risks at runtime. During engineering, the assembly system and its environment are modelled in Dassault's 3D Experience platform. Known or "native" hazard potentials can be adopted from the assets used in the setup. The safety engineer is responsible for the identification of new hazard potentials based on the combination of the assets in the particular setup. A knowledge graph representation of

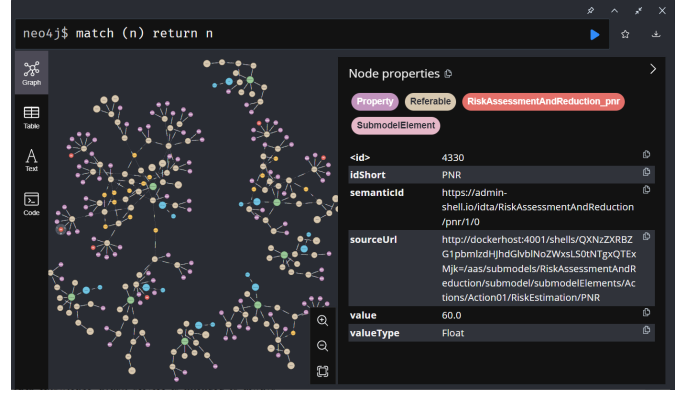


Fig. 4. A knowledge graph in Neo4j based on the contents of AAS and submodels.

the modelled hazards is then used to identify hazards that are not yet treated appropriately. Here, we have been using the Neo4j-based knowledge graph illustrated in Fig 4. For the identification of hazards we use the Neo4j graph query language Cypher.

All information during the engineering process is stored in a common database and is accessible through the AAS in terms of the safety submodel. This information can be used to simulate hazardous situations in a simulation environment and, since they rely on the same interfaces, prepare a safe startup of the assembly system.

At runtime, a safety monitoring tracks if the hazard potentials modelled during engineering are always disarmed by the corresponding risk mitigating capabilities. Once a rule violation is detected, risk mitigating capabilities are initiated.

In the scenario, a worker is pressing sealing rings into the distributor housings of a gear box at an assembly work station. The press action is disarmed by a light barrier and deactivates the press when a worker enters the press area (crossing a light grid). Wearing mixed reality glasses, such a light grid can be made visible, which is useful during risk assessment. Such sensor data streams are described in a respective signal submodel of the work station.

Since workers are usually focused on their work, they are inattentive against hazards in their vicinity when they are outside their field of view. Such a hazard may occur from patrolling AGVs carrying out transportation missions. The pose, i.e. the location and the orientation, of an AGV together with its planned route are accessible via a respective signal submodel. Multimodal warning mechanisms that increase with the proximity of the hazard are used to inform the worker of a hazard. Fig. 5 shows such an alert containing the distance of the mobile robot with its direction to the worker. Thereby, accidents through spontaneous movements of the worker can be avoided. If no mixed reality glasses are worn, such information can be provided on traditional worker guidance systems or using smart watches.



Fig. 5. Alert in mixed reality glasses notifying the worker about AGVs in the vicinity.

III. CORE AAS-BASED SERVICE INFRASTRUCTURE

Our infrastructure is conceptually designed as a distributed microservice architecture that uses a set of AAS and associated submodels to uniformly represent asset-specific information.

A. Asset Administration Shells and Submodels

In order to reflect most relevant information aspects of our lab within an exhaustive digital twin with which typical I4.0 and Human Robot Collaboration (HRC) use-cases can be covered, all relevant tangible and non-tangible assets of the lab such as robots, multimodal input and output devices, sensors, products, as well as processes were represented as AAS and appropriate submodels. Whenever possible, we used the submodel specifications published by the Industrial Digital Twin Association (IDTA)² that were available at the time. In addition to rather immutable submodels, such Digital Nameplate (IDTA-02006), Technical Data (IDTA-02003), Control-Component Type (IDTA-02015) and Instance (IDTA-02016), a set of dynamic submodels whose contents might also change over time during production have been designed throughout several BaSys satellite projects.

Among these, we designed a *topology submodel* for describing the hierarchical structure of the production site. The concept of that submodel constitutes a modified version of the joint model [9] based on ANSI/ISA-88 (Physical Model, Process Model) and ANSI/ISA-95 (Equipment Model) with adequate level of detail. Fig. 6 depicts a description of the model in the AASX format. Therein, we follow concepts of ISA-88 and provide a detailed modeling of hierarchical relationships across multiple levels between different physical entities involved in the production process, such as enterprises (e.g. DFKI), sites (e.g. Power4Production), areas (e.g. MRK 4.0 Lab), work centers (Fig. 7) to work units (such as robots or work stations and machines) which then consist of several equipment modules (such as grippers, cameras, arms or shelves) again consisting of several control modules such as e.g. fingers and servo motors.

We model physical entities as self-managed Entity, i.e. with a reference to the respective AAS of the physical en-

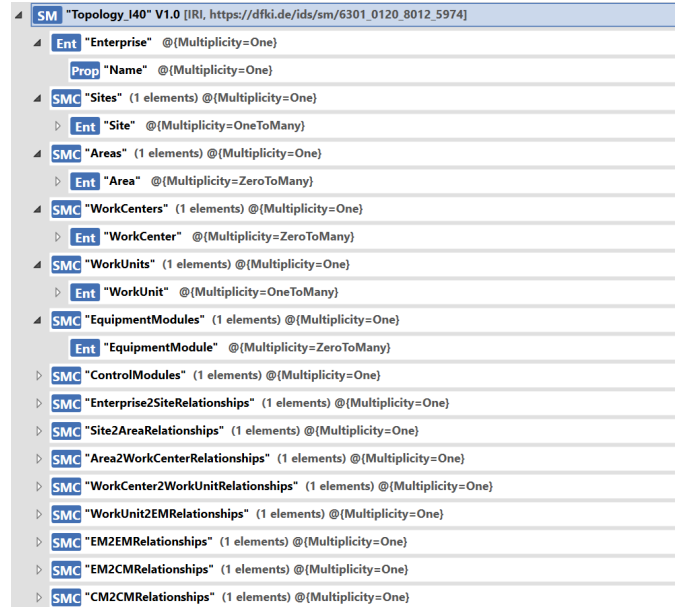


Fig. 6. AASX-based representation of our proposed submodel template for describing the topology of a I4.0 production site.

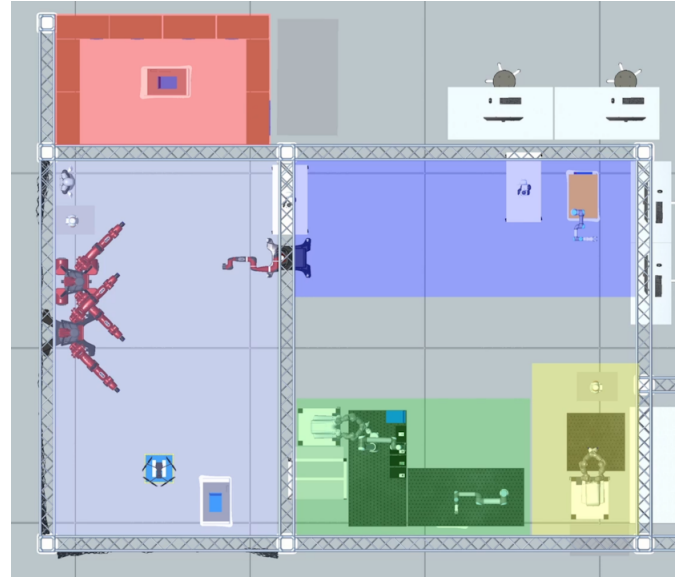


Fig. 7. Birds-eye perspective on a 3D representation of the lab highlighting different work centers: Warehouse (red), Quality Assurance-Center (yellow), Collaborative Assembly Line (green), Teach-In Center (dark blue)

tity and annotate their hierarchical relationships with submodel elements of type Relationship. E.g. an enterprise contains one or more sites, which themselves contain one or more areas containing an arbitrary number of work centers. Work units are considered to be of type LogisticsUnit (equipment used for transport, e.g. an AGV) or ProductionUnit (e.g. a single-arm robot) and are equipped with an AnnotatedRelationship element with a Type property (Contains, BelongsTo), which can be used to assign a WorkUnit to an arbitrary topology element,

²<https://industrialdigitaltwin.org/en/content-hub/submodels>

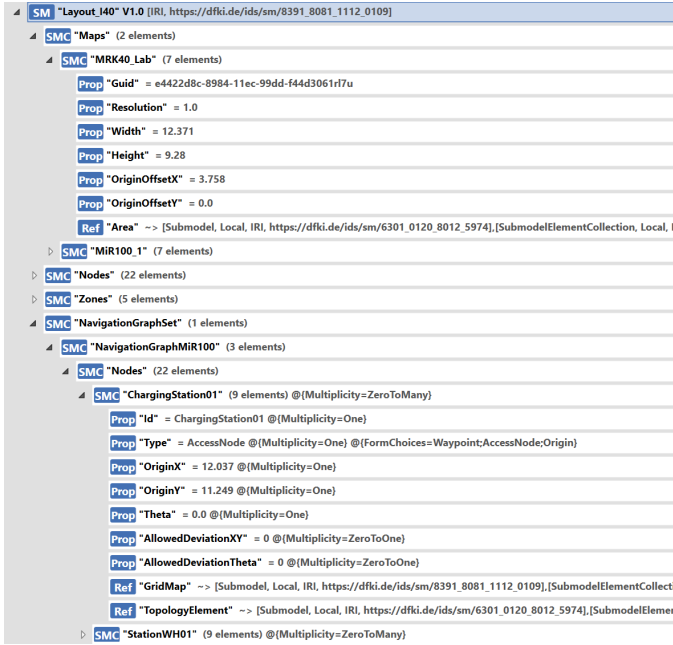


Fig. 8. AASX submodel describing the layout of a production site.

which introduces a certain flexibility for units which do not necessarily belong to a fixed work center such as AGVs. In this case, such a work unit could be assigned to a whole area or site.

While the proposed topology submodel exposes important information on hierarchical relationships of physical entities, we are still missing layout information by means of geometrical relationships between different areas, zones or stations and the descriptions of route networks, e.g. where an AGV can approach a certain work center, where it is allowed to drive, or information on possibilities to wait/park and charge. To this end, we designed a *layout submodel*, incorporating concepts of the VDA 5050 standard which describes the communication between a control system and AGVs. Fig. 8 depicts an overview.

A main geographical representation of a production environment is constituted by a two-dimensional (2D) grid-map consisting of a number of squared cells by means of a fixed resolution (meters/cell). As a map typically belongs to a topological area, we added a corresponding reference to that area. A zone describes a rectangular region within an area. Examples for zones are landing zones for drones, parking zones for mobile robots, specially annotated zones telling AGVs that the corresponding region is not to be accessed by them, i.e. forbidden, or, in contrast, should be considered as preferred when planning routes. Workspaces constitute another type of zone, e.g. one could define an approximated workspace on a table of a one-arm robot with a rectangular region. Zones can also describe a functional space, e.g. a region which contains several work units for assembly can group them within a collaborative assembly zone. A zone is described by a 2D origin in map coordinates (meters), a width (x)

and height (y) in meters and an angular deviation from the map's global x-Axis (theta) in degrees. Furthermore, a zone contains references to the corresponding map object and the topology object it belongs to. For a workspace, this would typically be a *WorkUnit* object while parking zones are generally associated to an *Area* element. In order to describe a route network which can be used by AGVs, we follow the notation of the VDA5050 standard and opt for a graph-like representation of navigation graphs consisting of nodes and relationships of these nodes (edges). With its type, a node represents either a *Waypoint*, or an *AccessNode* for a station for loading and unloading, or a *ChargingStation* and, similar to an area, it is geographically represented by a 2D coordinate and the deviation from the global x-Axis of the map (theta). Optionally, it defines an allowed deviation for an AGV from the intended position by means of a tolerance radius in meters. As defined in the VDA5050 standard, a node is considered to have been traversed by an AGV if the AGV passes the node within the defined radius. The same holds for the orientation by means of the property *AllowedDeviationTheta*. Similarly, as a zone, a node object holds references to its grid map and the associated topology element. Finally, edges (as binary, annotated relationships between nodes) describe a path segment, that can be traversed while following specific rules as intended by the annotation property. An orientation of 0 degrees for example always refers to traversing the edge with the front of the AGV facing in the direction of the edge. In addition, it is defined if rotations exceeding a certain threshold for corrective movements are allowed on an edge.

Control Components expose a set of skills that encapsulate real-time requirements and thus allow an external event-driven process orchestrator combining such skills into changeable production processes. Every work unit in the lab is represented by a full stack of *Control Component submodels*, i.e. a Type (IDTA-02015), an Instance (IDTA-02016) as well as a Control Component Interface submodel. The interface submodel maps the contents of the underlying OPC-UA information tree of the control component into a dedicated submodel allowing to control and query such a component via the REST-based submodel endpoint. For the implementation of control components, we use our Java-based open-source Control Component SDK³ that automatically generates an OPC-UA server as well as the Instance and Interface submodels. The type submodel is typically modeled by hand with the AASX package explorer.

In order to describe data structures and emission policies of important data streams of an asset, we refined a *signal submodel* originally designed in the BaSys satellite project BaSys-BDE-VWS depicted by Fig. 9. The model is meant to be of a generic form, such that not only technical operation data of production and logistics assets such as joint states of a robot arm, battery states, telemetry and positional data of AGVs but also organizational operation data like states of control components, processes, tasks or products can be represented.

³<https://github.com/dfkibasys/control-component>

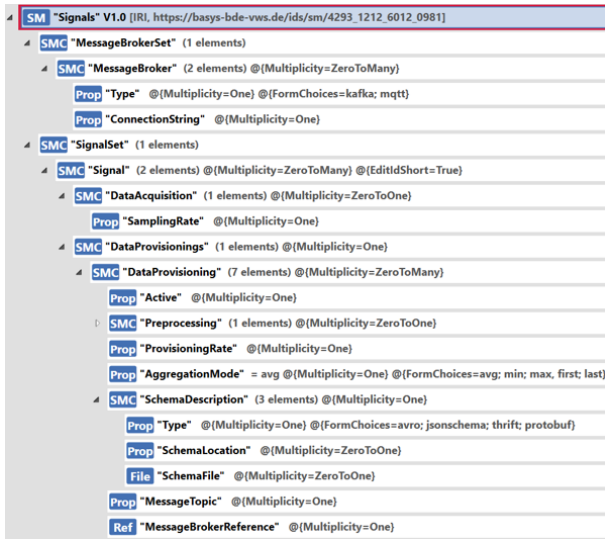


Fig. 9. AASX submodel template for describing signals provided as data streams.

The model defines a set of message brokers by which signals are distributed. Important properties of a message broker are its type, e.g. an MQTT or Apache Kafka Broker and its protocol, address and port by means of a connection string. Signals of an asset are collected in a signal-set where a signal is basically represented by its way of acquisition and provisioning. To keep it simple, we restricted the complex field of data acquisition to the frequency of measurements of a signal in terms of the sampling rate. Similarly, the data provisioning of a signal is characterized by a provisioning rate. Information about if and how a signal has been pre-processed before its provisioning is provided by the `Preprocessing` element which consists of several filter sets which themselves contain an arbitrary number of filters. A filter is described by its type, in order to reveal its semantics, like e.g. Gaussian2D, Median1D or three-dimensional high-pass filters which are then specified by an associated configuration string by means of a JSON descriptor. Also, a type of aggregation of the signal can be specified by means of an aggregation mode. In order to provide a description of the underlying data structure of a signal, we added a schema description with a specific type like JSON schema, Apache Avro, Apache Thrift or Google's Protobuf, a schema location and an optional descriptor file associated to the data type. Finally, a signal contains a reference element to the message broker it is associated with.

Fig. 10 depicts our proposal for modelling production processes in I4.0 by means of a *process submodel*.

As before, we opt for a BOM-like representation of processes. Following the notion of ISA-88, a process may include a number of subordinate processes, which we call `SubProcesses` which themselves may include several operations (`Tasks`) consisting of zero or more actions. According to ISA-88, a work unit may (while offering needed capabilities) realize a whole subprocess, a task or atomic action, which provides a certain flexibility in modelling. Task and Action

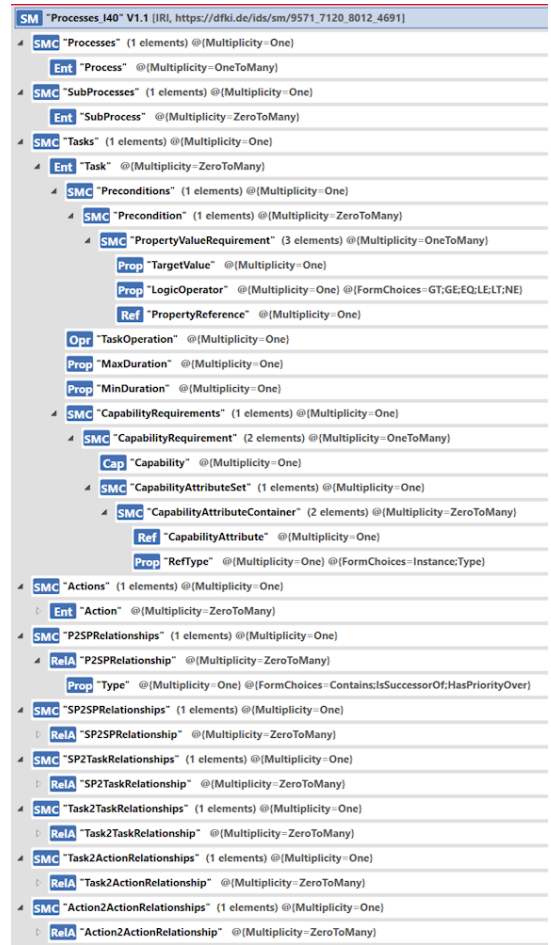


Fig. 10. AASX submodel template for describing different kinds of processes in I4.0

elements have the same structure in the proposed submodel. A task is basically represented by an `Operation` element with a set of input and output parameters (referencing e.g. elements of a BOM submodel of a product type AAS), a maximum and minimum duration property (i.e. the maximum duration reserved for the successful execution of the task, until the task is to be considered as delayed and the duration at least needed for the successful execution of the task, respectively) and a set of capability requirements. A capability requirement follows the notion of a capability descriptor in the *capability submodel* (IDTA-02020). Hence, a capability requirement consists of the capability itself (represented by a `Capability` element) and a set of capability attributes. An example would be the task of joining two different object parts of certain object type together where a corresponding production unit with capability `join` and related attributes for the associated object types would be needed and to be identified by considering the topology submodel and references therein. In addition, a task may have a set of preconditions by means of `PropertyValueRequirement` elements. A property value requirement specifies a target value, a reference to a property value and an associated logical operator which

describes the binary relationship of the reference value to the target value by means of equality, inequality, etc. With this approach, the execution of a task can be bound to a necessary precondition related to e.g. the output variables of another task operation. Besides process, subprocess, task and action entities, we define corresponding annotated relationships between them, in order to model containment and priority relationships. A task can be annotated as direct successor of another task or subprocess or more general be annotated as to be executed before another task or subprocess. Note that this kind of modelling is not meant to model an exact work plan with concrete workflows between tasks as it is common practice in BPMN modelling for example. The proposed process submodel is meant to enable a planning system to derive such a work plan description with assigned resources by interpreting said relationships.

This submodel can be used for describing a task and capability-based manufacturing process of a product as part of the type AAS of the product. Typically, it is accompanied by a BOM submodel of the product, describing the structure of parts of the product.

B. REST-based Service Components and Microservices

At the core of each microservice architecture there is a message broker that delivers messages according to a publish-subscribe model thus allowing a loose coupling of a sender and potentially many receivers. We use Apache Kafka as message broker due to its scalability and fast message delivery. Kafka also retains messages for a configurable amount of time, allowing microservices to build up their own internal context and update it if they temporarily drop out. However, for some applications it is easier and sometimes the only possible way to have an additional MQTT broker at hand, e.g. for sending MQTT events over Websockets to a browser-based dashboard or for receiving status updates from a PLC via MQTT. We use the EMQX broker for that. We bridge all events from MQTT to Kafka in order to retain these messages and having the complete picture in Kafka. In the other direction we only forward necessary messages. Fig. 11 illustrates the architecture and includes additional components that are discussed in the following. When performing, e.g. time-based event aggregation or complex event processing in a distributed system, it is important to synchronize the system clocks of all compute nodes and integrate the creation timestamps of the messages into the messages. We achieve this with a central time server and by defining all messages first in the Apache Avro interface description language where the base message already contains a property for the creation timestamp. A nice side-effect is that we can generate the message classes in multiple programming languages (currently Java, C#, Python, and TypeScript) for use in the different components in Fig. 11.

Most important service components are the **AAS Repository** and the **AAS Registry**. At the moment, we use the off-the-shelf components provided by Eclipse BaSyx, i.e. the AAS Repository with the BaSyx REST-API and the AAS Registry in V3.0

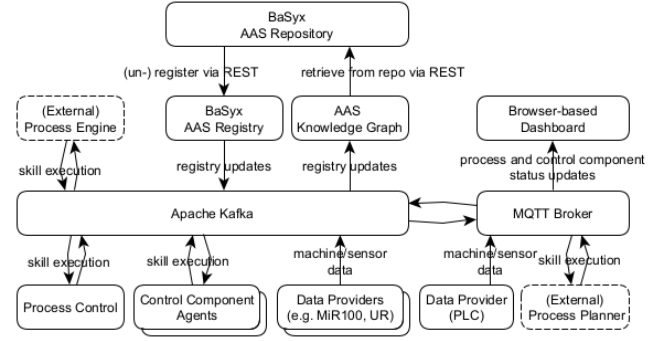


Fig. 11. Microservice architecture of the AAS-based service infrastructure

of the IDTA specification in combination with a compatibility layer for seamless interoperability with the repository. As the registry was contributed to BaSyx by us, we were able to extend the API in order to provide a lightweight search functionality over AAS and submodel descriptors. Also, the registry implementation is capable of sending update events to Apache Kafka whenever a new AAS or submodel has been registered or existing ones were unregistered. We extensively use this feature to dynamically react on changes, e.g. on the shopfloor when a new control component became available or to update the knowledge graph.

The **AAS Knowledge Graph** service is intended to always reflect the structure and thus also partly the contents of registered AAS and submodels. Whenever a new submodel is registered it is structurally integrated into the knowledge graph as a sub-graph that is connected to a node representing an AAS. Additionally, references in the submodel are resolved if possible by explicitly introducing an edge between the reference node and the target node in the graph. If a submodel is unregistered, the whole sub-graph is automatically removed. As mentioned, the knowledge graph service relies on Neo4j in the back-end. Each node stores a pointer to its source, i.e., a URL. Additionally, the nodes are multi-labeled taking into account both the AAS meta model hierarchy as well as the semantic IDs of the submodel elements. This makes writing Cypher queries for information retrieval much more elegant.

The **Process Control** service processes requests to perform skills of control components. Such a request can e.g. originate from an executing BPMN process in the Camunda process engine or from an adaptive online planner as presented in Section II-A. In order to integrate a control component in the infrastructure, a **Control Component Agent** that automatically spawns when a control component registers its submodels closes the gap between Apache Kafka and OPC-UA.

Data Providers are typically small scripts that acquire (sensor) data from an asset and provide this data in a pre-defined (Avro) structure either event-based or at a configurable frequency to a message topic in Apache Kafka or the MQTT broker. This data stream has to be described in the signal submodel of the respective asset, either manually or (if possible) by the data provider itself. If the data stream is

originally published to MQTT and then forwarded to Kafka, these two data streams will be described individually in the signal submodel. Data providers are highly custom as they are bound to a certain back-end communication technology and a certain programming language. As an example, for retrieving the battery status of our MiR100 AMRs, we access the underlying Robot Operating System (ROS) environment whereas the joint states of a UR robot is retrieved from an XML-RPC server running on the teach panel. Both data providers are implemented in Python and publish to Apache Kafka. Another data provider is realized on a PLC and publishes status updates to MQTT via a dedicated function block.

Part of the infrastructure is also a browser-based *Dashboard* illustrated in the top-right part of Fig. 3 that lets one start and monitor BPMN processes and shows the current status of control components. The list of available control components is retrieved from the AAS registry. Status updates of the control components are received via MQTT over WebSockets. The execution of a skill can be stopped and the control component itself can be reset. This is accomplished by REST calls to the respective control component interface submodel.

C. Container-based Deployment

All described components and microservices (except the data providers) are available as images on DockerHub⁴. With our Docker-Compose stacks⁵, we deploy the whole infrastructure in a Docker environment with just a single command on the command line. Custom AASX-files can be dropped in the data/aasx folder that is mounted into the repository container for hosting. Likewise, custom BPMN processes can be dropped in the data/processes folder for an automatic deployment into Camunda. We typically use this approach for spawning a development environment on a developer's computer as well as for setting up the actual lab infrastructure. A dedicated entry page provides easy access to each component's user interface.

IV. CONCLUSION AND FUTURE WORK

In this paper, we gave a detailed look into our AAS-based service infrastructure and its components in terms of AAS, submodel templates and their application in the context of our lab as well as core software services. We also explained how these components interact. The infrastructure is extendable and is published as open source on GitHub. Directly usable components are available on DockerHub. The practical use cases implemented on the infrastructure as well as on available hardware in our lab prove the usability of the infrastructure in an industrial context. Along with this, we believe that transferability and applicability to similar shopfloors is possible. Operators of other I4.0 testbeds as well as SMEs in particular can benefit from this.

The wheel of innovation with regard to the AAS and submodel specifications is accelerating. We will carefully analyze which newly released specifications to adopt, e.g.

hierarchical structures (IDTA-02011) in combination with topological information. We plan also to improve individual services and eliminate known limitations. For instance, the knowledge graph service currently cannot react to structural or value changes in a submodel. This will be tackled once the infrastructure has been fully upgraded to the latest stable version 3.0 of the AAS specification. Then, we will also evaluate the use of the recently released BaSyx DataBridge.

In the AIQUAMA project we are currently seamlessly integrating new sensors like cameras or scales together with corresponding AI services for hand tracking running on Nvidia Jetson edge devices into the infrastructure. This also entails the switch to Kubernetes to enable an actual distribution of services on multiple nodes in a computing cluster. In the frame of the European RICAIP project, we plan to connect our lab with I4.0 testbeds of partners and introduce additional components for an agent-based production. Gaia-X and the EDC will also play a role here.

ACKNOWLEDGMENT

This work has been funded by the German Ministry for Education and Research (BMBF) in the context of the project BaSys4Transfer (01IS22089C). We thank our project partners SYNAOS GmbH and Engrotec GmbH & Co. KG for the extraordinary collaboration in our BaSys satellite projects BaSysnaos and BaSysSafe.

REFERENCES

- [1] IDTA, "Details of the Asset Administration Shell - Part 1," 2023. [Online]. Available: https://industrialdigitaltwin.org/wp-content/uploads/2023/04/IDTA-01001-3-0_SpecificationAssetAdministrationShell_Part1_Metamodel.pdf
- [2] —, "Details of the Asset Administration Shell - Part 2," 2023. [Online]. Available: https://industrialdigitaltwin.org/wp-content/uploads/2023/04/IDTA-01002-3-0_SpecificationAssetAdministrationShell_Part2_API.pdf
- [3] A. Luxenburger, J. Mohr, T. Spieldenner, D. Merkel, F. Espinosa, T. Schwartz, F. Reinicke, J. Ahlers, and M. Stoyke, "Augmented reality for human-robot cooperation in aircraft assembly," in *Proceeding of the International Conference on Artificial Intelligence and Virtual Reality (AIVR'19)*. IEEE, 2019, pp. 263–266.
- [4] S. Knoch, M. Hüster, M. Jost, J. Mohr, D. Merkel, A. Luxenburger, T. Schwartz, J.-J. Kim, D.-Y. Koh, and J. Park, "Wall-et: Assistance in supermarkets and warehouses through social cognitive robots," in *Proceedings of the International Conference on Artificial Intelligence and Virtual Reality (AIVR'22)*. IEEE, 2022, pp. 219–220.
- [5] C. Jacob, F. Espinosa, A. Luxenburger, D. Merkel, J. Mohr, T. Schwartz, N. Gajjar, and K. Rekik, "Digital twins for distributed collaborative work in shared production," in *Proceedings of the International Conference on Artificial Intelligence and Virtual Reality (AIVR'22)*. IEEE, 2022, pp. 210–212.
- [6] N. Kleer, M. Rekrut, J. Wolter, T. Schwartz, and M. Feld, "A multimodal teach-in approach to the pick-and-place problem in human-robot collaboration," in *Companion of the International Conference on Human-Robot Interaction (HRI'23)*. ACM, 2023, p. 81–85.
- [7] G. Schuh, R. Anderl, J. Gausemeier, M. ten Hompel, and W. Wahlster, Eds., *Industrie 4.0 Maturity Index: Managing the Digital Transformation of Companies*, ser. acatech Studie. München: Utz, 2017.
- [8] F. Schnicke, T. Kuhn, T. Klausmann, S. Grüner, and D. Porta, "Architecture blueprints for the application of the industry 4.0 asset administration shell," in *Proceedings of the International Conference on Emerging Technologies and Factory Automation (ETFA'22)*. IEEE Press, 2022, p. 1–8.
- [9] D. Gocheva, I. Batchkova, M. Dobrev, and K. Velez, "Domain ontology development for engineering support systems," in *International Conference AUTOMATICS AND INFORMATICS (ICAI'09)*, 2009, pp. III–9 – III–12.

⁴<https://hub.docker.com/u/eclipsebasysx>, <https://hub.docker.com/u/dfkibasys>

⁵<https://github.com/dfkibasys/docker>